КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет комп'ютерних наук та кібернетики
Кафедра теорії та технології програмування

Звіт до навчального проєкту № 1 з дисципліни «Методи паралельних обчислень» на тему «Паралельна побудова зворотного індексу»

Виконав студент 4-го курсу
Групи ТТП-42
Тимофій ЛЕЙЦИШИН

Київ — 2026

1. УМОВИ ПРОВЕДЕННЯ ЕКСПЕРИМЕНТУ

Конфігурацію обчислювальної системи та умови запуску наведено в табл. 1.

Таблиця 1 – Умови проведення експерименту

Параметр	**Значення**
Процесор	Apple M1 Max (ARM64): 10 фізичних ядер без SMT (10 логічних), з них 8 продуктивних (P, два кластери по 4 ядра) і 2 енергоефективні (E)
Кеш-пам’ять	L1 даних: 128 КБ на P-ядро, 64 КБ на E-ядро; L2: 12 МБ на P-кластер, 4 МБ на E-кластер; L3 відсутній, його роль виконує системний кеш (SLC) 48 МБ
Рядок кешу	128 байт (`sysctl hw.cachelinesize`; підтверджено дослідженням хибного розділення)
Оперативна пам’ять	32 ГБ LPDDR5
ОС і компілятор	macOS 26.6.2 (25G83); Go 1.26.2 darwin/arm64
Збирання	`go build -trimpath`; оптимізація та вбудовування функцій увімкнені за замовчуванням (аналог `-O2`), без `-race` і `-gcflags=-N -l`
Умови запуску	20.10.2026, 00:24–00:44; живлення від мережі, режим енергозбереження вимкнено, сторонні застосунки закрито; load average на початку вимірювань 3,4

Корпус містить 722 книги англійською мовою з Project Gutenberg, завантажені скриптом `scripts/corpus.sh` через інтерфейс масового завантаження (robot harvest). Обсяг корпусу становить 242 722 532 байти (231,5 МіБ), розмір файлів від 30 КБ до 2,4 МБ (медіана 260 КБ); усі файли мають коректне кодування UTF-8. Після нормалізації корпус містить 32 284 949 токенів і 495 761 різне слово. Розподіл частот відповідає закону Ципфа: на 20 найчастотніших слів припадає 26,75 % усіх входжень, зокрема 7,61 % на слово `the`.

2. МЕТОДИКА ВИМІРЮВАНЬ

- Вимірюваною величиною є час побудови індексу за монотонним годинником. Корпус зчитується в пам’ять до вимірювання, сортування і запис результату виконуються після нього. Час злиття у V4 входить до часу побудови і додатково вимірюється окремо.
- Кожна конфігурація (версія, P, K) вимірюється окремим запуском програми, який будує індекс шість разів: перша побудова є попередньою і не враховується, наступні п’ять є вимірюваними повтореннями. Перед кожним повторенням виконується збирання сміття, а індекс попереднього повторення не зберігається, тому всі повторення починаються з однаковим обсягом зайнятої пам’яті.
- Наводиться медіана п’яти повторень, розкид характеризують мінімальне і максимальне значення. Усі повторення записано у файл `results.csv` (`program, version, threads, k, run, seconds`). $S(P) = T(V0) / T(P)$, $E(P) = S(P) / P$.
- Вихідний файл кожного запуску побайтово порівнюється з результатом V0; розбіжностей не виявлено.
- Усі вимірювання виконує скрипт `scripts/final.sh`: $P \in \{1, 2, 4, 6, 8, 10\}$, для V3 K = 1024, залежність від K досліджено за P = 8.

3. ОПИС РЕАЛІЗАЦІЇ

Програму написано мовою Go без сторонніх бібліотек. Версії містяться в пакеті `internal/index` (файли `v0.go`–`v4.go`, `cas.go`, `syncmap.go`), програма `cmd/invindex` запускає будь-яку з них, а `cmd/falseshare` досліджує хибне розділення. Потоки виконання реалізовано горутинами, які середовище виконання розподіляє між потоками ОС (GOMAXPROCS = 10).

Еталонна функція нормалізації `Words` використовує стандартну бібліотеку: `bytes.FieldsFuncSeq` виділяє максимальні послідовності літер і цифр (`unicode.IsLetter`, `unicode.IsDigit`), будь-який інший символ, зокрема некоректна послідовність UTF-8, є роздільником; токени, коротші за три символи, відкидаються, решта переводиться в нижній регістр. Під час вимірювань використовується оптимізована функція `FastWords` (підрозд. 5.7), збіг її результатів з `Words` перевіряють модульні тести і фазинг-тестування.

Індекс є хеш-таблицею `map[string]*Entry`; `Entry` містить загальну кількість входжень слова `Total` і таблицю `Docs` «номер документа → кількість входжень». Одна операція оновлення відповідає одному входженню слова. Рядки вихідного файлу впорядковано за словом, записи в рядку за спаданням кількості, за

однакової кількості за зростанням номера документа; документи нумеруються з нуля в лексикографічному порядку шляхів.

Роботу розподілено за файлами: кожна з P горутин отримує номер наступного необробленого файлу зі спільного атомарного лічильника, що вирівнює навантаження за різного розміру файлів. Атомарна операція виконується один раз на файл і не стосується індексу. Реалізацію версій наведено в табл. 2.

Таблиця 2 – Реалізація версій програми

Версія	**Реалізація**
V0	один потік, один індекс
V1	спільна хеш-таблиця з методом ланцюжків без синхронізації (стандартна `map` за одночасного запису аварійно завершує програму). Для кожного слова є вузол загальної кількості і по вузлу на документ; після створення вузла потоки змінюють лише поля розміром у машинне слово, тому гонка призводить до втрати оновлень, але не пошкоджує пам'ять
V2	один індекс і одна блокувальна змінна `sync.Mutex`, яка захоплюється на час кожного оновлення
V3	K секцій з власними блокувальною змінною і хеш-таблицею; слово належить секції hash(слово) mod K. Секцію доповнено заповнювачем до 128 байт, щоб блокувальні змінні різних секцій не потрапляли в один рядок кешу
V4	кожна горутина будує локальний індекс без синхронізації, потім індекси зливаються: `v4seq` послідовно приєднує їх до першого (P − 1 крок), `v4tree` зливає попарно за схемою збалансованого дерева, паралельно на кожному рівні ($\lceil \log_2 P \rceil$ кроків)
`cas` (додатково)	таблиця V1, у якій обидві операції з гонкою атомарні: вузол додається операцією `CompareAndSwap` з повтором, лічильники збільшуються `atomic.Int64.Add`
`syncmap` (додатково)	`sync.Map` (конкурентне префіксне дерево хешів) з ключем «слово, документ» і атомарним лічильником як значенням

У програмі дослідження хибного розділення P горутин виконують по n інкрементів власного лічильника `int64`. Лічильники розміщено в масиві із заданою відстанню: 8 байт відповідає реалізації A (`long counters[P]`), 64 байти реалізації B (`struct { long value; char pad[56]; }`), додатково досліджено відстані 16–256 байт. Масив вирівняно на межу 128 байт; дизасемблювання підтверджує, що кожна ітерація читає і записує пам'ять.

4. РЕЗУЛЬТАТИ

Час побудови і прискорення наведено в табл. 3 і на рис. 1 (графіки 1 і 2). Час V0 становить 2,992 с (2,933–3,050 с).

Таблиця 3 – Медіана часу побудови індексу, с, і прискорення (у дужках)

P	**V2**	**V3 (K = 1024)**	**V4, послідовне**	**V4, дерево**	**`cas`**	**`syncmap`**
1	3,104 (0,96)	3,465 (0,86)	3,031 (0,99)	3,033 (0,99)	4,088 (0,73)	8,892 (0,34)
2	6,301 (0,47)	1,895 (1,58)	1,837 (1,63)	1,802 (1,66)	2,781 (1,08)	5,708 (0,52)
4	7,730 (0,39)	1,168 (2,56)	1,231 (2,43)	1,198 (2,50)	2,138 (1,40)	4,135 (0,72)
6	9,197 (0,33)	1,559 (1,92)	0,995 (3,01)	0,946 (3,16)	1,997 (1,50)	3,657 (0,82)
8	10,516 (0,28)	1,540 (1,94)	0,922 (3,25)	0,836 (3,58)	1,904 (1,57)	3,733 (0,80)
10	11,545 (0,26)	1,624 (1,84)	0,942 (3,18)	0,842 (3,55)	1,909 (1,57)	3,540 (0,85)

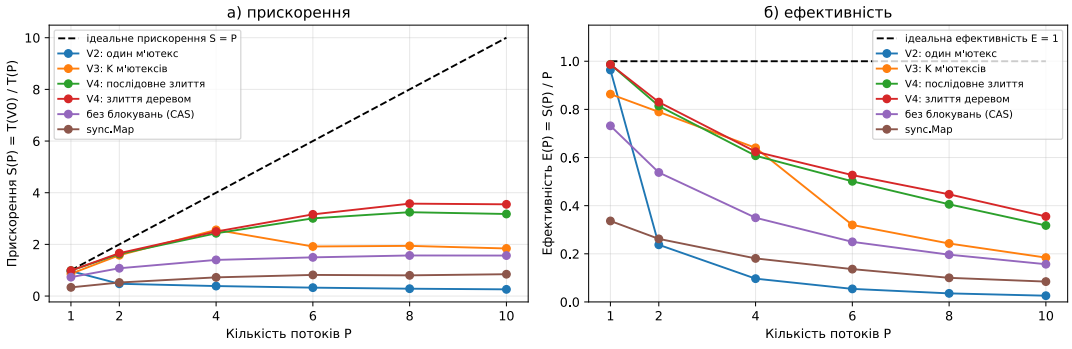


Рисунок 1 – Прискорення (а) та ефективність (б) залежно від кількості потоків

Результати десяти запусків V1 за P = 8 наведено в табл. 4; еталонна кількість входжень (V0) становить 32 284 949. Жоден запуск не дав правильного результату, і всі результати різні. Середня частка втрачених оновлень становить 0,12 % за P = 2, 0,52 % за P = 4, 2,33 % за P = 8 і 2,74 % за P = 10 (по 10 запусків).

Таблиця 4 – Втрачені оновлення у V1 за P = 8

Запуск	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Втрачено, тис.	785,2	746,8	800,5	802,5	727,6	808,2	717,9	597,0	749,9	789,3
Втрачено, %	2,43	2,31	2,48	2,49	2,25	2,50	2,22	1,85	2,32	2,45

Час V3 за P = 8 залежно від K наведено в табл. 5 і на рис. 2 (графік 3). Для кожного K наведено частку оновлень найбільш навантаженої секції і Σf^2, тобто ймовірність того, що два випадкові оновлення звертаються до однієї блокувальної змінної (f є часткою оновлень секції).

Таблиця 5 – Час V3 і навантаження блокувальних змінних залежно від K (P = 8)

K	**1**	**4**	**16**	**64**	**256**	**1024**	**4096**
Медіана, с	10,909	6,681	3,324	2,297	1,718	1,540	1,489
Мін.–макс., с	10,79–11,18	6,49–7,54	3,31–3,67	2,26–2,33	1,70–1,75	1,52–1,56	1,47–1,56
Найбільш навантажена секція, %	100	29,94	12,63	8,79	7,77	7,64	7,61
Σf^2	1	0,2570	0,0722	0,0245	0,0128	0,0102	0,0096

Контрольні вимірювання (скрипт `scripts/supplementary.sh`): на копії корпусу без 20 найчастотніших слів V3 за P = 4, 6 і 8 працює 0,979, 0,956 і 0,809 с (V0 на цьому корпусі 2,642 с, S(8) = 3,27); без заповнювача в секціях V3 за P = 8 повільніша на 26 % за K = 16 (4,017 с проти 3,193 с), на 19 % за K = 256 і на 12 % за K = 1024.

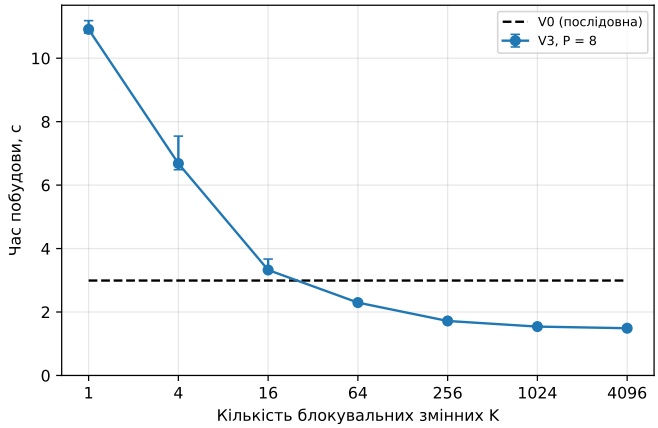


Рисунок 2 – Час виконання V3 залежно від K (P = 8)

Час злиття і частку злиття в загальному часі V4 зі злиттям деревом наведено в табл. 6.

Таблиця 6 – Злиття у V4

P	**2**	**4**	**6**	**8**	**10**
Послідовне злиття, с	0,165	0,296	0,382	0,451	0,503
Злиття деревом, с	0,167	0,285	0,345	0,368	0,407
Частка злиття (дерево)	9 %	24 %	36 %	44 %	48 %
Прискорення побудови локальних індексів	1,85	3,32	5,04	6,48	6,97

Оцінка обсягу роботи злиття за P = 8, отримана з індексу V0: на критичному шляху послідовна схема копіює 3 458 648 пар «документ – кількість» за 7 кроків, схема дерева 3 331 683 пари за 3 кроки (0,46, 0,94 і 1,93 млн). Максимальний обсяг пам’яті процесу становить 624 МБ для V0, 626 МБ для V3, 912 МБ для V4 з послідовним злиттям і 992 МБ зі злиттям деревом. Вісім локальних індексів разом містять 853 436 записів слів, а підсумковий 495 761.

Час для 10 млн інкрементів на потік, передбачених завданням, наведено на рис. 3, а (графік 4). За такої тривалості (3–9 мс) помітно впливає створення горутин, тому висновки зроблено за вимірюванням для 200 млн інкрементів (табл. 7, рис. 3, б).

Таблиця 7 – Медіана часу, мс, для 200 млн інкрементів на потік

Відстань, байт	**8 (A)**	**32**	**40**	**64 (B)**	**96**	**128**	**256**
P = 1	64,5	62,1	64,5	65,9	62,2	62,6	62,2
P = 2	257,9	257,7	258,3	73,4	72,7	72,0	67,8
P = 4	283,5	271,0	271,6	87,6	85,5	83,5	76,8
P = 8	337,3	314,0	311,8	283,3	121,3	87,5	87,3

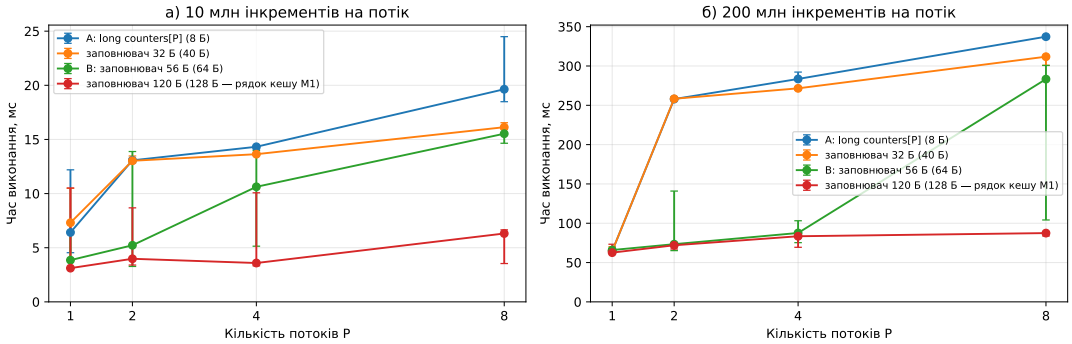


Рисунок 3 – Час виконання реалізацій A, B та інших відстаней між лічильниками для 10 млн (а) і 200 млн (б) інкрементів на потік

Значення метрики Карпа – Флатта e = (1/S − 1/P) / (1 − 1/P) наведено в табл. 8 і на рис. 4.

Таблиця 8 – Метрика Карпа – Флатта

P	**V2**	**V3**	**V4, послідовне**	**V4, дерево**	**`cas`**	**`syncmap`**
2	3,21	0,267	0,228	0,204	0,859	2,82
4	3,11	0,187	0,215	0,201	0,620	1,51
6	3,49	0,425	0,199	0,180	0,601	1,27
8	3,87	0,446	0,209	0,177	0,585	1,28
10	4,18	0,492	0,239	0,202	0,598	1,20

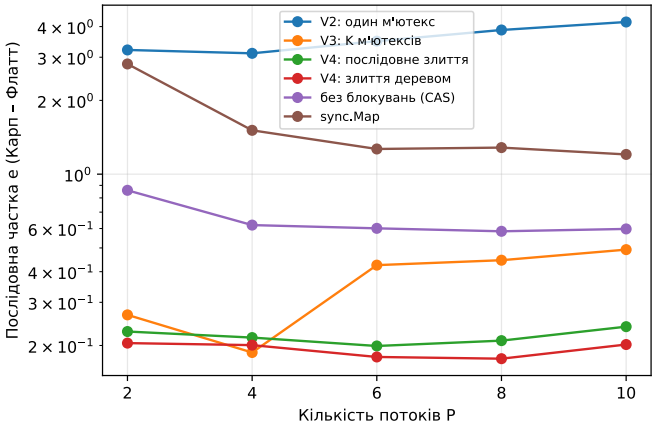


Рисунок 4 – Метрика Карпа – Флатта (логарифмічна шкала)

Розподіл процесорного часу за даними профілювальника (`-cpuprofile`, `go tool pprof`) наведено в табл. 9.

Таблиця 9 – Розподіл процесорного часу за даними профілювання

Версія	**Основні ділянки коду**
V0	`madvise` (отримання пам’яті від ОС) 24 %, вставлення в хеш-таблицю 20 %, нормалізація 16 %
V2, P = 8	простій ядер у планувальнику (`usleep`) 49 %, переведення горутин в очікування і пробудження (`pthread_cond_wait`, `pthread_cond_signal`) 35 %, повільний шлях `Lock` 4 %, хеш-таблиця менше 2 %
V3, P = 8	`usleep` 67 %, `pthread_cond_wait` 9 %, вставлення в хеш-таблицю 4 %
V4 (дерево), P = 8	`madvise` 50 %, вставлення в хеш-таблицю 8 %, нормалізація 6 %
`cas`, P = 8	`madvise` 33 %, пошук і вставлення в таблиці 13 %, нормалізація 9 %
`syncmap`, P = 8	`madvise` 36 %, обхід дерева `HashTrieMap` 10 %, хешування ключів інтерфейсного типу 9 %

На 20 книгах корпусу швидкість нормалізації становить 125 МБ/с для `Words` і 332 МБ/с для `FastWords` (у 2,65 раза більше). Час V0 становить 4,399 с з `Words` і 2,992 с з `FastWords` (у 1,47 раза менше), V4 зі злиттям деревом за P = 8 відповідно 1,034 і 0,836 с (у 1,24 раза менше).

5. АНАЛІЗ РЕЗУЛЬТАТІВ

5.1 Версії V0 і V1

Близько чверті часу V0 займає системний виклик `madvise`, через який ОС надає пам'ять у міру зростання купи (індекс займає близько 400 МБ), п'яту частину займає вставлення в хеш-таблицю, решту нормалізація і пошук.

Гонка даних у V1 виникає в рядку `v1.go:31` (`nd.count++`), який компілюється в три інструкції: читання (`MOVD 32(R1), R2`), збільшення (`ADD $1, R2, R2`) і запис (`MOVD R2, 32(R1)`). Якщо другий потік читає лічильник між читанням і записом першого, обидва записують однакове значення, і одне оновлення втрачається. Лічильник документа змінює лише один потік, а загальну кількість слова змінюють усі, тому в контрольному запуску 94,4 % із 733 927 втрачених оновлень припали на 20 найчастотніших слів (433 241 на `the`), а для 1 746 слів загальна кількість перестала дорівнювати сумі за документами. Друге місце гонки, додавання вузла в комірку (`v1.go:35`: читання голови ланцюжка, створення вузла, запис нової голови), через велику кількість комірок виникає рідко: у тому самому запуску втрачено один запис документа. Обидва рядки виявляє детектор гонок Go. Частка втрат зростає з P, бо зростає кількість одночасних звертань до лічильників частих слів.

5.2 Версія V2

Під захистом блокувальної змінної виконується вся робота з індексом, а паралельно лише нормалізація (менше п'ятої частини часу), тому за законом Амдала $S \le 1{,}2$. Фактично за $P \ge 2$ V2 повільніша за V0 (10,5 с проти 3,0 с за P = 8): горутина, яка після короткого активного очікування не захопила `sync.Mutex`, переводиться в стан очікування і пробуджується після його звільнення, а вільні ядра простоюють у планувальнику. Корисна робота становить менше 5 % процесорного часу; крім того, рядки кешу з блокувальною змінною і зміненими записами індексу щоразу передаються між ядрами. Метрика Карпа – Флатта більша за 1 і зростає з P: накладні витрати перевищують корисну роботу.

5.3 Версія V3

Зі зростанням K величина Σf^2 спадає з 1 до 0,0102, і час зменшується. Після $K \approx 1024$ час практично не змінюється: K = 4096 дає лише 3 % (розкиди перетинаються), Σf^2 зменшується на 6 % проти 20 % між K = 256 і 1024. Межу визначає розподіл частот: усі оновлення слова захищає одна блокувальна змінна, тому найбільш навантажена секція за будь-якого K отримує не менше 7,61 % оновлень (частка `the`), а вже за K = 256 її частка становить 7,77 %. Подальше збільшення K розподіляє лише рідкісні слова, оновлення яких майже не конфліктують. Саме тому графік 3 спадає, поки K розносить 20 частих слів (26,75 % оновлень) по різних секціях, і виходить на плато після $K \approx 1024$.

Прискорення V3 зростає до P = 4 (2,56) і спадає за $P \ge 6$ (1,92–1,94), а e зростає з 0,19 до 0,45, тобто прискорення обмежують накладні витрати на синхронізацію, а не послідовна частина. За профілем 67 % процесорного часу займає простій у планувальнику: горутини чекають звільнення блокувальних змінних найбільш навантажених секцій. Контрольне вимірювання це підтверджує: без 20 найчастотніших слів V3 масштабується до восьми потоків (S = 3,27) без спаду між P = 4 і 6. Спад на повному корпусі, імовірно, зумовлений будовою M1 Max: до чотирьох потоків виконуються в одному P-кластері зі спільним L2, а за $P > 4$ рядки кешу з блокувальними змінними і записами частих слів передаються між кластерами через SLC, що значно повільніше. Без заповнювача вісім блокувальних змінних потрапляють в один рядок кешу, і через хибне розділення V3 сповільнюється на 12–26 %.

5.4 Версія V4

Побудова локальних індексів масштабується майже лінійно (S = 6,48 за P = 8, E = 0,81), бо потоки не мають спільних даних. Замість синхронізації з'являються інші накладні витрати: злиття (44 % часу за P = 8), отримання пам'яті для P локальних індексів (`madvise` займає половину процесорного часу; 912–992 МБ проти 624 МБ у V0, бо часті слова дублюються в кожному локальному індексі) та очікування найповільнішого потоку перед злиттям.

Злиття деревом швидше за послідовне лише на 18 % (0,368 с проти 0,451 с за P = 8), хоча має 3 кроки замість 7. На k-му рівні дерева кожен індекс уже містить дані 2^(k−1) локальних індексів, тому вартість рівнів зростає як 1 : 2 : 4, і для даних, що не повторюються в різних потоках, критичний шлях дерева становить P − 1 = 7 одиниць, як і в послідовної схеми. Теоретична оцінка це підтверджує (3,33 млн проти 3,46 млн скопійованих пар). Перевага дерева зумовлена паралельним об'єднанням частих слів, спільних для всіх потоків, на нижніх рівнях і кращим розміщенням менших хеш-таблиць нижніх рівнів у кеші. За P = 10 додаються два повільніші E-ядра, і час не зменшується.

5.5 Версія без блокувань і версія на основі sync.Map

У версії `cas` немає блокувальних змінних і простою в планувальнику, проте за P = 8 вона повільніша (1,90 с) за V3 (1,54 с) і V4 (0,84 с). Уже за P = 1 вона на 37 % повільніша за V0 через власну хеш-таблицю, атомарні операції на кожен токен, окремий вузол для кожної пари «слово, документ» і перетворення таблиці на індекс наприкінці. Масштабування обмежують ті самі часті слова: рядок кешу з лічильником `the`, який атомарно збільшують усі ядра, постійно передається між ними (істинне, а не хибне розділення), тому e ≈ 0,6 за будь-якого P. Версія `syncmap` найповільніша з коректних (8,9 с за P = 1, 3,5 с за P = 10): ключі й значення `sync.Map` мають тип `any`, тому для кожного входження створюється рядок і виділяється пам'ять під інтерфейсний ключ, а хешування такого ключа й обхід дерева повільніші, ніж у типізованій `map`. `sync.Map` оптимізована для ключів, які записуються один раз і багато разів читаються, а побудова індексу постійно записує ті самі ключі.

5.6 Хибне розділення

Один потік виконує 200 млн інкрементів приблизно за 63 мс за будь-якого розміщення: рядок кешу з лічильником перебуває в L1 у стані *Modified*, інкремент займає близько такту. Протокол когерентності працює з цілими рядками: для запису ядро отримує виключний доступ до рядка, а копії в кешах інших ядер стають недійсними. Якщо лічильники різних потоків лежать в одному рядку, кожен запис змушує інше ядро повторно завантажувати рядок, тому реалізація A повільніша, ніж за P = 1, у 4 рази за P = 2 і в 5,2 раза за P = 8, хоча спільних даних немає. Хибне розділення зникає лише за відстані 128 байт (за P = 8 час для 128 і 256 байт однаковий), що відповідає розміру рядка кешу M1 Max. Реалізація B розрахована на 64-байтний рядок x86 і на M1 недостатня: за P = 8 вона лише на 16 % швидша за A, бо сусідні лічильники лежать в одному 128-байтному рядку. За P = 2–4 відстань 64 байти майже не сповільнює роботу, імовірно, тому що потоки виконуються в одному кластері зі спільним L2. Заповнювач 32 байти (відстань 40 байт) хибного розділення практично не зменшує: 258 мс за P = 2, як у A, і 312 мс за P = 8 проти 88 мс за 128 байт. Отже, заповнювач має дорівнювати розміру рядка кешу за вирахуванням розміру лічильника: 120 байт для M1 і 56 байт для 64-байтного рядка.

5.7 Закон Амдала й оптимізація нормалізації

Для V4 зі злиттям деревом e майже не залежить від P (0,18–0,20), тобто відповідає справжній послідовній частині: злиттю і послідовним частинам отримання пам'яті. За законом Амдала $S(\infty) = 1/e \approx 5{,}6$, а для P = 8 оцінка становить близько 3,6, що відповідає виміряному S(8) = 3,58. Тому найвища ефективність за восьми потоків, 0,45 у V4 зі злиттям деревом, є очікуваною: побудова локальних індексів має ефективність 0,81, а злиття займає 44 % часу. Ефективність інших версій становить від 0,04 (V2) до 0,24 (V3) через синхронізацію на частих словах. Для V2 і V3 e зростає з P, отже, прискорення обмежують накладні витрати на синхронізацію: прогноз для V3 за значенням e для P = 2 (S(8) = 2,79) перевищує фактичне 1,94. Для V4 фактичне S(8) = 3,58 більше за прогноз 3,29, бо частина накладних витрат за P = 2 від P не залежить. Відхилення від ідеального прискорення зумовлюють злиття, синхронізація на частих словах, отримання пам'яті від ОС, два кластери P-ядер і повільніші E-ядра за P = 10.

`FastWords` обробляє ASCII-текст по 8 байтів за ітерацію методом SWAR (SIMD within a register): вісім символів класифікуються кількома арифметичними операціями над 64-бітним словом, а слова з малих літер ASCII повертаються без копіювання. Нормалізація прискорюється у 2,65 раза, V0 у 1,47 раза, а V4 за P = 8 лише в 1,24 раза: нормалізація добре розпаралелюється, тому її прискорення найбільше впливає на послідовну версію.

6. ВИСНОВКИ

– Продуктивність визначає не кількість потоків, а частота звертань потоків до спільних даних. Найшвидша версія V4 зі злиттям деревом (S = 3,58 за P = 8): потоки працюють без спільних даних, а синхронізацію замінює злиття.
– Одна блокувальна змінна на весь індекс (V2) робить оновлення послідовними і додає витрати на очікування та пробудження потоків, тому V2 повільніша за послідовну версію у 2,1–3,9 раза.
– Секціоноване блокування (V3) ефективне, доки збільшення K розподіляє часті слова між секціями; за природного розподілу частот кілька найчастотніших слів обмежують прискорення за будь-якого K.
– Відмова від блокувальних змінних (`cas`) не усуває конкуренції за лічильники частих слів, а універсальна конкурентна структура (`sync.Map`) повільніша за спеціалізовану.
– Розміщення даних у пам'яті суттєво впливає на продуктивність: хибне розділення сповільнює роботу в 4–5 разів, а рядок кешу M1 Max (128 байт) удвічі більший, ніж передбачає реалізація B.

7. ДЖЕРЕЛА ТА ІНСТРУМЕНТИ

– Project Gutenberg. Robot access policy: https://www.gutenberg.org/policy/robot_access.html; дзеркало PGLAF: https://gutenberg.pglaf.org.
– The Go Programming Language Specification; The Go Memory Model: https://go.dev/ref/mem; документація стандартної бібліотеки Go (`sync`, `sync/atomic`, `hash/maphash`, `bytes`, `unicode`, `runtime/pprof`, `testing`).
– Інструменти: Go 1.26.2 (`go test -race`, фазинг-тестування, `go tool pprof`, `go tool objdump`). Запозиченого коду немає.